

~~Server/server_gui.py~~

```
# server_gui.py
# it is GUI for ai_learning_server_gui.py of file ' ' (§ )

import threading
import tkinter as tk
import sys
import os
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from tkinter import ttk, scrolledtext, messagebox

from Server.server_core import AILearningServer
from env_utils import DEFAULT_HOST, DEFAULT_PORT

class ServerGUI:
    def __init__(self, root: tk.Tk):
        self.root = root
        self.root.title("AI Learning Server GUI")
        self.root.geometry("900x700")

        self.server: AILearningServer | None = None
        self.server_thread: threading.Thread | None = None
        self.connected_clients = {} # it is a dict of connected clients, key is client id, value is client object

        self._create_widgets()

    def _create_widgets(self):
        top = ttk.LabelFrame(self.root, text="Server Control")
        top.pack(fill="x", padx=10, pady=10)

        ttk.Label(top, text="Host IP:").grid(row=0, column=0, padx=5, pady=5, sticky="w")
        self.host_entry = ttk.Entry(top, width=18)
        self.host_entry.insert(0, DEFAULT_HOST)
        self.host_entry.grid(row=0, column=1, padx=5, pady=5)

        ttk.Label(top, text="Port:").grid(row=0, column=2, padx=5, pady=5, sticky="w")
        self.port_entry = ttk.Entry(top, width=8)
        self.port_entry.insert(0, str(DEFAULT_PORT))
        self.port_entry.grid(row=0, column=3, padx=5, pady=5)

        self.start_btn = ttk.Button(top, text="Start Server", command=self.start_server)
        self.start_btn.grid(row=0, column=4, padx=5, pady=5)

        self.stop_btn = ttk.Button(top, text="Stop Server", command=self.stop_server, state="disabled")
        self.stop_btn.grid(row=0, column=5, padx=5, pady=5)

        self.status_label = ttk.Label(top, text="Server Status: ", foreground="red")
        self.status_label.grid(row=1, column=0, columnspan=6, padx=5, pady=5, sticky="w")

        # log frame
        log_frame = ttk.LabelFrame(self.root, text="Log")
        log_frame.pack(fill="both", expand=True, padx=10, pady=10)

        self.log_text = scrolledtext.ScrolledText(
```

```

Server/server_gui.py
    log_text.pack(height=20, width=80, state="disabled", font=("Consolas", 9)
    )
    self.log_text.pack(fill="both", expand=True, padx=5, pady=5)

    # í .° ...ì .ì ,í , ì °£†° ì °‡.
    client_frame = ttk.LabelFrame(self.root, text="ì °£†° ì í .° ...ì .ì ,í ,")
    client_frame.pack(fill="x", padx=10, pady=(0, 10))

    self.client_listbox = tk.Listbox(client_frame, height=4, font=("Consolas", 9))
    self.client_listbox.pack(fill="x", padx=5, pady=5)

    self.client_count_label = ttk.Label(client_frame, text="ì ì í .° ...ì .ì ,í ,: 0° " ")
    self.client_count_label.pack(side="left", padx=5, pady=5)

# --- ì °† ì ì . ---

def start_server(self):
    if self.server_thread and self.server_thread.is_alive():
        messagebox.showinfo("ì °f...", "ì .°fl, ì °† £° ì /í ì / ì ° ° /.")
        return

    host = self.host_entry.get().strip() or DEFAULT_HOST
    try:
        port = int(self.port_entry.get().strip() or DEFAULT_PORT)
    except ValueError:
        messagebox.showerror("ì /°¥ ", "í <í ,° ì «ì ì <ì ... í '° ° /.")
        return

    self.server = AILearningServer(host=host, port=port, log_callback=self._log)

    def run_server():
        self._log(f"[ì °† ] ì ì : {host}:{port}")
        self._set_status(running=True, host=host, port=port)
        try:
            self.server.start()
        finally:
            self._set_status(running=False)
            self._log("[ì °† ] ì / ì § ° ¢")
            self._update_client_list()

    self.server_thread = threading.Thread(target=run_server, daemon=True)
    self.server_thread.start()

    self.start_btn.config(state="disabled")
    self.stop_btn.config(state="normal")

def stop_server(self):
    if self.server:
        self.server.stop()
        self.start_btn.config(state="normal")
        self.stop_btn.config(state="disabled")

# --- UI ì ° ì .í , ---

def _log(self, msg: str):

```

Server/servergui.py

```
self.log_text.config(state="normal")
self.log_text.insert("end", msg + "\n")
self.log_text.see("end")
self.log_text.config(state="disabled")

# í .° ...ì .ì ,í , ì °£†° /ì ¢ °£ °' ì ì § í ì -
if "[ì °£†°] í .° ...ì .ì ,í ,:" in msg:
    parts = msg.split("í .° ...ì .ì ,í ,:")
    if len(parts) > 1:
        client_id = parts[1].strip()
        self.connected_clients[client_id] = client_id
        self._update_client_list()
elif "[ì ¢ °£ ] í .° ...ì .ì ,í , ì °£†° ì ¢ °£ : " in msg:
    parts = msg.split("í .° ...ì .ì ,í , ì °£†° ì ¢ °£ : ")
    if len(parts) > 1:
        client_id = parts[1].strip()
        if client_id in self.connected_clients:
            del self.connected_clients[client_id]
        self._update_client_list()

self.root.after(0, append)

def _update_client_list(self):
    """ì °£†°° í .° ...ì .ì ,í , °'°ì ì °°ì .í ,"""
    def update():
        self.client_listbox.delete(0, tk.END)
        for client_id in self.connected_clients.values():
            self.client_listbox.insert(tk.END, client_id)
        self.client_count_label.config(text=f"ì ì í .° ...ì .ì ,í ,: {len(self.connected_client

self.root.after(0, update)

def _set_status(self, running: bool, host: str | None = None, port: int | None = None):
    def update():
        if running:
            self.status_label.config(
                text=f"ì í : ì /í ì / ({host}:{port})", foreground="green"
            )
        else:
            self.status_label.config(text="ì í : ì / ì § ° ¢", foreground="red")

    self.root.after(0, update)

def main():
    root = tk.Tk()
    app = ServerGUI(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```